orc · user guide

orc is a development orchestrator for Claude Code. You describe a change in plain words. orc turns it into a ticket, builds it in an isolated copy of your project, runs the checks itself, has a fresh-context reviewer judge the result, merges one commit you can undo, and shows you a card with what changed and what it cost. It stops for exactly three reasons: a secret, a destructive command, or a production action.

It is built for people who are not engineers yet. The engineering discipline (specs, tests, review, receipts, lessons) runs underneath; you see questions you can answer and results you can try.

Status: pre-release. It has run end to end on Windows 11 with Node 24 and Claude Code 2.1. Expect rough edges and tell us about them.

---

1. Install

You need three things on your machine: Claude Code 2.1 or newer, Node 20 or newer, and git. Then orc itself, which is a Claude Code plugin. If you already use Claude Code from a terminal and have Node and git, skip to 1.2.

1.1 Claude Code, Node and git

An account. Claude Code needs a Claude Pro, Max, Team or Enterprise subscription, or a Claude Console account with API credits. The free claude.ai plan does not include it. Pick one at claude.com/pricing before you install.

Install Claude Code. Open a terminal and run the line for your system. On Windows, a prompt that starts with `PS` is PowerShell and one that starts with `C:` is CMD; use the matching line.

```
macOS, Linux, WSL:   curl -fsSL https://claude.ai/install.sh | bash
Windows PowerShell:  irm https://claude.ai/install.ps1 | iex
Windows CMD:         curl -fsSL https://claude.ai/install.cmd -o install.cmd &&
install.cmd && del install.cmd
```

The installer prints nothing while it downloads. When it finishes, open a new terminal window and run `claude --version`. A working install prints a version such as `2.1.317 (Claude Code)`. If the shell says `claude` is not found, the install folder is not on your PATH yet; the page at code.claude.com/docs/en/troubleshoot-install covers that and every other install error. Homebrew (`brew install --cask claude-code`) and WinGet (`winget install Anthropic.ClaudeCode`) also work but do not update themselves. The native install updates in the background; `claude update` forces an update and `claude doctor` checks the install.

Sign in. Run `claude` inside any folder. The first start opens your browser to log in with the account above. After that your login is remembered. Type `/exit` to leave.

Node. orc's command line and its hooks are one JavaScript file, so Node must be installed even though Claude Code itself does not need it. Download the LTS version from nodejs.org and run the installer with the defaults. `node --version` should print v20 or higher.

git. orc builds in git worktrees and merges with git, so git is required. On macOS, run `git --version` once and accept the prompt to install the command line tools. On Linux, install the `git` package. On Windows, install Git for Windows from git-scm.com with the defaults; it also gives Claude Code the Bash shell it prefers. Check with `git --version`.

Your project must be a git repository. If it is not yet, run `git init` in its folder once. `orc init` tells you if any of the above is missing.

1.2 orc

Inside Claude Code, in the project you want to work on:

```
/plugin marketplace add LevFurmanov/startup_starterpack
/plugin install orc@orc
/orc init
```

To test unreleased changes, or if GitHub is not reachable from your machine, point the marketplace at a local clone instead: `/plugin marketplace add C:\path\to\startup_starterpack`.

`orc init` detects your check command (npm test, pytest, go test, cargo test), writes `orc.yaml`, creates the `vault/` folder, adds deny rules for secret files to `.claude/settings.json`, and appends a short block to `CLAUDE.md` that routes build requests to orc. Nothing else in your project changes.

To use `orc` from a terminal as well, clone the repository (`git clone https://github.com/LevFurmanov/startup_starterpack`) and add its `plugins/orc/bin` folder to your PATH. It contains `orc` (shell) and `orc.cmd` (Windows). Everything the plugin does is one file, `bin/orc.mjs`, with no dependencies.

---

2. Your first ticket

In Claude Code:

```
/orc add a greet function that returns "Hello, <name>!" and cover it with a test
```

What happens, in order:

1. orc creates ticket T-001 and runs the scout, a cheap read-only agent that lists the files involved, estimates the size and the hours, and says how confident it is.
2. If the scout is confident (green), you are asked nothing. Otherwise you get at most three questions, phrased as outcomes, each with a recommended answer. Typing `go` accepts the recommendations.
3. Claude writes the spec: goal, acceptance criteria, what must not happen, how to verify.
4. The engine takes over. It builds, checks, reviews and merges in the background while you keep talking.
5. You get a card:

```
┌ T-006 merged ──────────────────────────────────────────────────────┐
│ What changed   Added titleCase(text) in text.mjs, which uppercases… │
│ Try it         1. npm test                                         │
│ Checks         pass · 1 receipt(s) run by orc · revert check pass  │
│ Diff           4 files changed, 43 insertions(+), 1 deletion(-)    │
│ Review         1 of 1 criteria verified · 3 finding(s), 0 blocking │
│ Cost           $0.46 · scout haiku 3k · worker sonnet 2k · reviewer… │
│ Undo           orc undo T-006                                      │
└────────────────────────────────────────────────────────────────────┘
```

From a terminal, `orc run "add a greet function ..."` does the same with default answers and no questions.

---

3. How it works

There are two halves. The conversation runs in your Claude Code session: ticket, scout, questions, spec. The engine is code: `orc run T-001` drives every step after that and launches each agent as its own headless Claude process with the model, turn cap, time cap, dollar budget, tool list and security guard set by orc. Agents do the work; orc owns state, git, checks, review routing and merge.

Step	Who	Model	Reads	Writes
intake	your session	your session	your request	ticket T-###
scout	engine	haiku	repo, lessons	scout report, size, confidence
questions	your session	your session	scout report	decisions on the ticket
spec	your session	your session	answers	spec under 60 lines
dispatch	engine	none	spec	branch orc/T-###, worktree
build	engine	sonnet, opus for L	spec, rules, last handoff	commits on the branch
check	engine	none	worktree	receipts, revert check
review	engine	worker's model, opus for L	spec, diff, receipts	findings with confidence
fix	engine	same as build	findings, handoff	commits, at most 2 rounds
merge	engine	none	branch	one squashed Conventional Commits commit on main
learn	engine	none	work and review reports	lessons
report	engine	none	records	done card, dashboard

Sizes decide the ceremony. The scout proposes, orc enforces.

Size	Means	Worker	Review
S	one file, text or config, no new behavior	sonnet	checks only
M	up to five files, one behavior	sonnet	bug judge
L	anything else, or auth, payments, deletion, migrations, secrets	opus	bug, proof and simplify judges
chore	renames, formatting, dependencies	haiku	checks only

Anything the scout estimates above two hours is split into dependent tickets automatically; you build them one after another.

---

4. What orc never trusts

The research behind orc (docs/research/05-orchestration-practices.md) kept repeating one finding: nothing an agent writes is a result. Tests, "done" claims, lessons and status are claims until the orchestrator has run something itself. orc is built around that.

Receipts. After the build, orc runs the check command and every `$` line of the spec's "How to verify" itself, in the worktree, and stores a receipt per command: commit, command, exit code, output. The reviewer and the done card show receipts, not the worker's quotes.

Revert check. When the ticket added or changed tests, orc reverts the production change, runs the check again, and restores. Tests that still pass without the change are flagged as inert, because they prove nothing.

Test ratchet. Tests, fixtures and any file the check command runs are read-only for the worker if they existed before the ticket. New test files are welcome. If an old test is wrong, the worker reports it and stops; you decide, then `orc ticket allow-tests T-001 <path>` logs the exception.

Judges, not a reviewer. Each review pass is its own fresh process with one lens. The bug judge checks each criterion and hunts for the failure the worker did not test. The proof judge audits the receipts. The simplify judge may only propose deletions and reuse, never new abstractions, and never blocks. Every judge is told what cheating looks like: special-cased inputs, hard-coded outputs, tests on mocks, edited or skipped tests, changed check configuration, `process.exit(0)` escapes. Findings carry a confidence; only 0.7 and above cause a fix round. A judge that cannot tell says unknown instead of pass.

Stopping is success. The worker is told that reporting a flawed spec, a wrong test or a broken environment beats working around it. The ticket goes to `needs-you` with the reason in plain words.

Stall detector. The same tool call three times in a row gets a warning, six times stops the run. A worker also has a wall-clock cap of twice the minutes for its size. A cap hit writes a handoff ledger and re-dispatches once in a fresh process before asking you.

Handoff, never resume. Every round ends with `.orc/tickets/T-001.handoff.json`: goal, done-when, files changed, assumptions, unresolved items, check result. The next round is a fresh process that reads it. orc never resumes a session into its own mistakes.

Clean history. Every commit follows Conventional Commits (conventionalcommits.org). The worker's commits are checked by the guard: `type(scope): short description` under 72 characters, with a `Refs: T-001` footer; anything else is rejected before it runs. The merge squashes the ticket into one commit built from the ticket itself: the type from the ticket type (feat, fix, chore), the scope from the files changed, the acceptance criteria as the body, and `Refs`, `Reviewed-by` and `Receipts` footers. `orc undo` adds a `revert:` commit. Set `merge_strategy: ff` in `orc.yaml` to keep the worker's individual commits instead.

Three pauses. Secret files (`.env*`, `*key*`, `*secret*`, `*credential*`) can never be read, printed or written by an agent; when a task needs a secret, the ticket names the variable and where it goes and hands that step to you. Destructive commands (force push, hard reset, `rm -rf`, dropping tables) and production commands (deploys, publishes, ssh) pause the ticket. Everything else runs without approval, and every merge is one commit `orc undo T-001` reverts.

---

5. Where things live

After `orc init` your project has:

```
orc.yaml                    configuration, edit freely
vault/                      the words, Obsidian-friendly
  Home.md                   overview page
  Board.md                  kanban as text
  Dashboard.html            single-file dashboard: board, tickets, cost, lessons, health
  Decisions.md              every answer you gave, across tickets
  Tickets/T-001-*.md        the spec you and Claude edit, under a header orc renders
  Tickets/T-001-*.log.md    readable event log
  Reports/T-001/            scout.md, checks.md, review-1-bug.md, work.md
  Lessons/L-001-*.md        rules learned in this project
  _templates/ticket.md      the spec template, yours to change
.orc/
  tickets/T-001.json        the ticket record: status, size, cost, models, files
  tickets/T-001.events.jsonl   append-only events
  tickets/T-001.receipts.json  every check orc ran, with commit and output
  tickets/T-001.handoff.json   the ledger between rounds
  state.json, cache/, wt/, stall/   local: not committed
.claude/
  settings.json             deny rules for secret files
  rules/orc-<scope>.md      short negative rules, loaded by path
CLAUDE.md                   a short block that routes requests to /orc
```

State is JSON, words are markdown. The records in `.orc/tickets` are the truth and are committed with the project, so a clone carries the history. The vault file keeps the spec text; its header is rendered by orc and never read back, so editing it by hand changes nothing. Open `vault/` in Obsidian for a readable view; open `vault/Dashboard.html` in a browser for the dashboard (`orc dashboard --open`).

Postgres, optional. For several people or machines on one project, a hosted dashboard, or SQL over cost and review data, orc can mirror the same records into Postgres. One database is one project; there are no tenants. The schema (`plugins/orc/storage/schema.sql`, 17 tables, 2 views) groups into tasks, task state (append-only events with a current-state view), sessions and turns, sources and reports, and maintenance. Set `ORC_DATABASE_URL` in your shell, never in a file an agent can read, then `orc store postgres` applies

the schema. Agents never touch the store; they read a rendered spec and write through orc commands, so switching stores changes no prompt. Mirroring and import are not finished in this release; `orc store status` tells you what is on.

---

6. Lessons

When a review finds a bug class or the worker learns something, a one-sentence rule is proposed. orc validates it (one sentence, starts with Never, Always, Do not, Only, Avoid or Prefer, has a path scope), rejects anything else, and files it as `hypothesis`. Seen a second time it becomes `verified` and is promoted into `.claude/rules/orc-<scope>.md`, where Claude loads it only when editing matching files. A lesson with a check command, violated three times, becomes an enforced check that `orc check` runs. Rules untriggered for sixty days stay on disk but leave the prompts. At most twenty rules reach one worker prompt. `orc lessons reject L-003` marks a lesson a mistake and it never comes back. `/orc-skill-creator <scope>` turns a scope's lessons into a reusable skill.

---

7. Commands

Inside Claude Code you mostly type `/orc <request>`. Every command below also works from a terminal.

Command	Does
`orc init [--store vault or postgres]`	set up the project
`orc doctor`	check node, git, claude, config, worktrees
`orc explain [topic]`	print how a part works; topics: overview, loop, install, files, models, storage, security, proof, lessons, dashboard, cost, lifecycle, agents, hooks, commands, levels
`orc run T-001`	the engine: scout if needed, build, check, review, fix, merge, done card
`orc run "what you want"`	create a ticket from a sentence and run it with default answers
`orc new "<title>" [--size S or M or L] [--type feature or fix or chore]`	create a ticket
`orc ticket list`	all tickets with status, size, cost
`orc ticket show T-001`	the spec
`orc ticket set T-001 key=value`	change a field, for example `summary="..."`
`orc ticket log T-001`	the event log
`orc ticket allow-tests T-001 <glob>`	let the worker edit an existing test, logged
`orc ready`	tickets whose dependencies are merged
`orc needs-you`	tickets waiting on you, with the reason

`orc resume T-001`	after you handled what it needed; then `orc run T-001`
`orc close T-001 [--failed] [--reason "..."]`	close without merging, removes the worktree
`orc undo T-001`	revert the merged commit and open a redo ticket
`orc status`	in flight, needs-you, merged this week, ready
`orc done T-001`	the done card
`orc board`	regenerate Board.md, Home.md, Decisions.md, Dashboard.html
`orc dashboard --open`	the same, then open the dashboard
`orc cost [T-001 or --week]`	cost per ticket, per role and model
`orc models suggest`	per model and size: rounds, cost, undone rate, and a suggestion
`orc lessons list [--all]`	lessons with tier, provenance, triggers, scope
`orc lessons add --rule "Never ..." --scope <glob> [--check cmd]`	file a lesson by hand
`orc lessons add --from vault/Reports/T-001/work.md`	file the candidates in a report
`orc lessons reject L-003`	veto; never re-added
`orc lessons tier L-003 reference or guide or enforce`	move a lesson between tiers
`orc lessons skill <scope> --name <name>`	generate a skill from a scope's lessons
`orc store status or migrate or postgres`	the optional Postgres store

The single steps the engine runs, for when you want to drive by hand or watch one stage:

Command	Does
`orc scout T-001 --run`	run the scout and record size, files, confidence, hours
`orc scout T-001` then `--record`	write the scout prompt; record a report written by any agent
`orc dispatch T-001 [--model opus]`	create branch and worktree, write the build prompt
`orc check T-001`	run the checks, keep receipts, run the revert check
`orc review T-001 [--pass bug]` then `--record`	write the judge prompts; record their reports
`orc merge T-001`	rebase, re-check, squash into one conventional commit, remove the worktree

Skills the plugin installs: `/orc` (the conversation half), `/orc-skill-creator` (lessons into a skill), and ten background skills that agents and control load on their own: orc-questions, orc-spec, orc-scout, orc-build,

orc-review, orc-done-card, orc-fix, orc-smoke, orc-explain, orc-status, orc-undo.

---

8. orc.yaml

Every section has a comment. The ones you are likely to touch:

```
check: npm test              # what must pass before merge; orc runs it itself
models:
  worker: { S: sonnet, M: sonnet, L: opus, chore: haiku }
  reviewer: { S: worker, M: worker, L: opus }   # worker = same model, fresh process
review:
  M: [bug]
  L: [bug, proof, simplify]
  max_fix_rounds: 2
  min_confidence: 0.7
caps:
  turns: { worker: { S: 25, M: 25, L: 40 } }
  minutes: { S: 10, M: 20, L: 40 }           # worker gets twice this
  budget_usd: { S: 2, M: 8, L: 25 }          # a backstop, not the main stop
  max_hours: 2                               # scout splits above this
  agents: auto                               # agents at once: from your RAM and cores,
or a number
machine:
  ram_per_agent_gb: 1                        # what one agent needs; raised by the peaks
orc measures
  reserve_gb: 4                              # kept for the OS, your browser and your own
session
merge_strategy: squash                       # or ff to keep the worker's commits
forced_L: [auth, login, password, payment, billing, stripe, delete, migration, secret,
token]
security:
  secrets, destructive, production           # the three pause lists
  tests, check_files                         # what the ratchet protects, what the
reviewer sees
```

Model aliases resolve to the current generation. Pin full ids when a fixed model matters.

---

9. When it stops, and what it costs

`needs-you` is the only state that waits for you. `orc needs-you` prints the reason in plain words: a secret to set, a wrong test to decide on, a destructive command to run yourself, a spec that cannot be built as written, or a worker that hit its caps three times. Handle it, then `orc resume T-001` and `orc run T-001`.

Costs from the live runs on a small repository, with the default models:

Ticket	Scout	Worker	Reviewer	Total
greet function, one fix round	haiku, 1 cent	sonnet, 32 cents	opus, 36 cents	$0.69
slugify with tests	haiku, 1 cent	sonnet, 21 cents	opus, 42 cents	$0.64
titleCase with tests	haiku, 1 cent	sonnet, 13 cents	opus, 32 cents	$0.46

Each ticket took about three minutes of agent time. The reviewer costs as much as the build on small tickets, which is why the default reviewer for size M is now the worker's model at high effort; `orc models suggest` shows whether that holds on your project.

RAM. Each agent is its own Claude process: about 0.3 GB resident on a short run, up to about 1 GB on a long one, plus whatever your test command needs while it runs. orc limits how many agents run at once from your machine: the smallest of RAM (total minus a 4 GB reserve, divided by 1 GB per agent), cores minus two, and a ceiling of six. On a laptop with 16 GB and 20 cores that is six; on 8 GB it is four. `orc doctor` prints the number with the arithmetic, `orc status` shows how many are running, and every run records the real peak of each agent's process tree: the done card shows it per agent, `orc doctor` per role, and the estimate rises automatically when a run peaks higher. Set `caps.agents` in `orc.yaml` to pin a number; `1` means strictly one agent at a time. The limit covers both the engine's headless agents and any agents your Claude Code session starts, and when free memory drops below what one agent needs the next one waits.

---

10. Known limits

- One ticket builds at a time. Parallel tickets (level 1) are designed, not built.
- Postgres applies the schema only; mirroring the vault into it is not finished.
- No post-merge CI watch and no notifications yet; `orc status` and the dashboard are the signal.
- Windows is the tested platform. macOS and Linux should work (the CLI is plain Node and git) but have not been run end to end.
- The plugin has been proven through the terminal engine; the in-session `/orc` conversation is the part you are testing.
